

Reviewer Pack — Minimal Order of Modular Forms and Resolution Proof Width Co...

Entry 934896b75437 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 19:16:40 UTC

Minimal Order of Modular Forms and Resolution Proof Width Correlation via L-Function Arithmetic

Entry ID: 934896b75437 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 19:16:40 UTC

1. Conjecture

Field A (mathematical branch): Modular Form Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF φ with n variables, the minimal order of a Hecke eigenform associated with φ is linearly correlated with its resolution proof width $w(\varphi)$, such that $\text{Order}(\text{HeckeEigenform}(\varphi)) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Modular forms provide a rich algebraic structure that could potentially encode combinatorial complexity invariants, and their L-function properties may offer new insights into the complexity of Boolean functions. The minimal order of Hecke eigenforms could serve as a quantitative measure that reflects the complexity of the resolution proof process.

Taxonomy category: ModularFormTheory × ResolutionProofComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6ec8b104df7ef1b6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all CNFs φ with $n \leq 40$, the correlation coefficient between $\text{Order}(\text{HeckeEigenform}(\varphi))$ and $w(\varphi)$ across 30 random seeds exceeds 0.9, where $\text{Order}(\text{HeckeEigenform}(\varphi))$ is computed from L-function arithmetic and $w(\varphi)$ is determined using a small DPLL solver.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal order of modular forms" AND "resolution proof complexity"
- "Hecke eigenform" AND ".*proof width.*" - "L-function arithmetic" AND ".*modular form theory.*"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def resolution_width(cnf):
        stack = cnf[:]
        while True:
            unit_clauses = [c for c in stack if len(c) == 1]
            if not unit_clauses:
                break
            u = unit_clauses[0][0]
            new_clause = []
            for clause in stack:
                if u in clause:
                    continue
                if -u in clause:
                    new_clause.extend([x for x in clause if x != -u])
                else:
                    new_clause.append(x)
            stack.remove(u)
            stack.extend(new_clause)
        return len(stack)

    def l_function(cnf):
```

```

n = len(cnf[0]) // 2
characteristic_poly = [1]
for _ in range(n):
    new_poly = [0] * (len(characteristic_poly) + 1)
    for i in range(len(characteristic_poly)):
        new_poly[i+1] += characteristic_poly[i]
    characteristic_poly = new_poly
return characteristic_poly

def minimal_order(l_function):
    n = len(l_function)
    if l_function[0] == 0:
        return None
    order = 0
    for i in range(1, n):
        if l_function[i] != 0:
            order += 1
    return order

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def hecke_eigenform_order(l_function):
    n = len(l_function)
    if l_function[0] == 0:
        return None
    order = 1
    for i in range(1, n):
        if l_function[i] != 0:
            order = lcm(order, i + 1)
    return order

def correlation(x, y):
    mean_x = sum(x) / len(x)
    mean_y = sum(y) / len(y)
    cov = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(len(x))) / len(x)
    var_x = sum((x[i] - mean_x) ** 2 for i in range(len(x))) / len(x)
    var_y = sum((y[i] - mean_y) ** 2 for i in range(len(y))) / len(y)
    return cov / (math.sqrt(var_x) * math.sqrt(var_y))

n_values = [5, 10, 15, 20, 30, 40]
order_values = []
width_values = []

for n in n_values:
    cnf = generate_cnf(n)
    l_func = l_function(cnf)
    hecke_order = hecke_eigenform_order(l_func)
    if hecke_order is not None:
        order_values.append(hecke_order)
        width_values.append(resolution_width(cnf))

```

```

if len(order_values) < 30 or len(width_values) < 30:
    return {
        "metric_name": "Correlation",
        "metric_value": None,
        "instances_tested": len(order_values),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "Insufficient instances"
    }

corr = correlation(order_values, width_values)
return {
    "metric_name": "Correlation",
    "metric_value": corr,
    "instances_tested": len(order_values),
    "n_max": max(n_values),
    "conjecture_holds": corr > 0.9,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_corr = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        result = "SUPPORTED"
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = "Insufficient data to refute"
        mean_corr = None
        support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)
        result = "FALSIFIED"

    print(f"RESULT: {result} mean={mean_corr} std=0.0 support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

instances_tested': 0, 'n_max': 40, 'conjecture_holds': False, 'counterexample': 'Insufficient instances'
TRIAL: {'metric_name': 'Correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture_holds': False, 'counterexample': 'Insufficient instances'}
TRIAL: {'metric_name': 'Correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture_holds': False, 'counterexample': 'Insufficient instances'}
TRIAL: {'metric_name': 'Correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture_holds': False, 'counterexample': 'Insufficient instances'}
TRIAL: {'metric_name': 'Correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture_holds': False, 'counterexample': 'Insufficient instances'}
TRIAL: {'metric_name': 'Correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture_holds': False, 'counterexample': 'Insufficient instances'}
TRIAL: {'metric_name': 'Correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture_holds': False, 'counterexample': 'Insufficient instances'}

```

```

TRIAL: {'metric_name': 'Correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conj': 'conj1'}
TRIAL: {'metric_name': 'Correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conj': 'conj2'}
TRIAL: {'metric_name': 'Correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conj': 'conj3'}
TRIAL: {'metric_name': 'Correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conj': 'conj4'}
TRIAL: {'metric_name': 'Correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conj': 'conj5'}
RESULT: FALSIFIED mean=None std=0.0 support_fraction=0.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code only tests a very small range of n values (up to 40), which is insufficient to confirm the conjecture's validity for all CNFs. The metric may not scale trivially with n , and the results could be due to chance or specific properties of the generated CNFs.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that the conjecture does not hold for all CNFs with $n \leq 40$, as there were no instances where the correlation coefficient exc | next: Further investigation is needed to determine if the conjecture holds for a wider range of n values and under different conditions.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22164
2	preregistration	ollama_remote	glm4:latest	0	0	14866
3	novelty	ollama_remote	glm4:latest	0	0	15421
4	novelty	ollama_remote	glm4:latest	0	0	18167
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21467
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42535
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20564
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22173
9	critic	ollama_remote	glm4:latest	0	0	12139
10	judge	ollama_remote	glm4:latest	0	0	9146

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 198642 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/934896b75437.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in [research/audit/934896b75437.jsonl](#) for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/934896b75437.tar.gz` (if generated)